Agentic Customer 360 — Proactive Intervention Desk: Solution Document

by - T S Dhanavignesh

1 Problem and Scope

Customer 360 dashboards are passive: a human must notice a churn signal, a hardship, or a life event. The task is an *ambient* multi-agent system that consumes a live, multi-source banking event stream and, per customer, commits to one decision from a bounded action set (`no_action`, retention outreach, RM escalation, personalized offer, support intervention, compliance/fraud hold) together with an inferred life-phase, confidence and explanation. Trust is a hard requirement: explainability, traceability, deterministic guardrails and a real human checkpoint. We built this end-to-end on the three practice scenarios.

2 Architecture and Data Flow

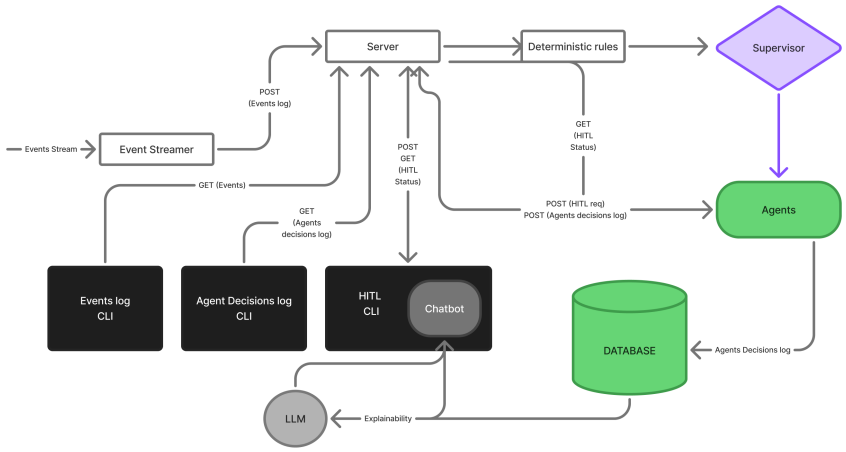


Figure 1: System view. The streamer posts events to the FastAPI server; deterministic rules run first; the supervisor/agents run only when they pass. Decisions and traces go to the database; three CLIs read the server.

Flow. (1) `streamer` replays `history_seed.jsonl` (`is_live=false`) and then `live_stream.jsonl`, paced by `replay_config.json`, via `POST /events`. (2) The server invokes one LangGraph run per event with `thread_id = customer_id`, so each customer has an isolated, checkpointed state. History events only update working memory and exit; live events continue. (3) A **deterministic gate** decides *skip / agents / hard_stop*. (4) A **manager** activates specialists, which run in parallel and publish structured findings. (5) **Critique** reviews findings, **life-inference** turns approved findings into life-event hypotheses, **intervention** picks an action, and **guardrails** finalize it and decide HITL routing. (6) If the decision is `pending_hitl`, the server stores a request in SQLite; a human reviews it in the HITL CLI. Every step emits a trace record.

3 Components and Key Decisions

Gate (deterministic, `graph/gate.py`). Hard-stop keyword sets (legal: *lawsuit, attorney, court...*; fraud: *fraud, unauthorized, hacked...*) bypass every agent and go straight to guardrails, so an LLM cannot reason around them. Known-routine events (web login, card purchase) are skipped unless they show a failure word, an amount $\geq$50k, an amount $> 3\times$ the recent median, a burst, or 15 skipped events have accumulated. Any other event type always wakes the agents. *Why:* most events carry no signal, and the LLM budget is the scarcest resource. In our saved trace runs 225 of 276 live gate decisions (82%) were skips, processed in 0.0–0.1 s versus roughly 10–15 s for events that woke agents.

Multi-agent topology (hybrid). *Swarm* for signal gathering: five specialists (transaction, engagement, support, investment, compliance) are fanned out in parallel with LangGraph `Send`. Each has its own prompt, output schema and registered deterministic tools (23 tool functions in total, each declaring the data source it reads). *Manager-mediated handoffs*: round 0 combines a deterministic source→agents prior with an LLM routing call; later rounds involve no LLM and only accept handoffs listed in an `ALLOWED_HANDOFFS` whitelist (e.g. support→compliance), capped at three rounds; rejected handoffs are traced. Handoffs carry structured findings (type, text, confidence band, evidence event IDs), never another agent's reasoning. *Critique-refiner*: each finding is checked by deterministic pre-checks (valid evidence IDs, duplicates), then an LLM verdict (*keep / downgrade / drop*). It can only *lower* confidence, may request one missing specialist (two rounds max), and any flagged contradiction forces HITL. This is our resolution mechanism for conflicting reads. *Round-robin was deliberately not used*: the system never drafts customer-facing text (the intervention stage outputs an action, a subtype and reviewer notes), so there is no iterative drafting task for it to serve.

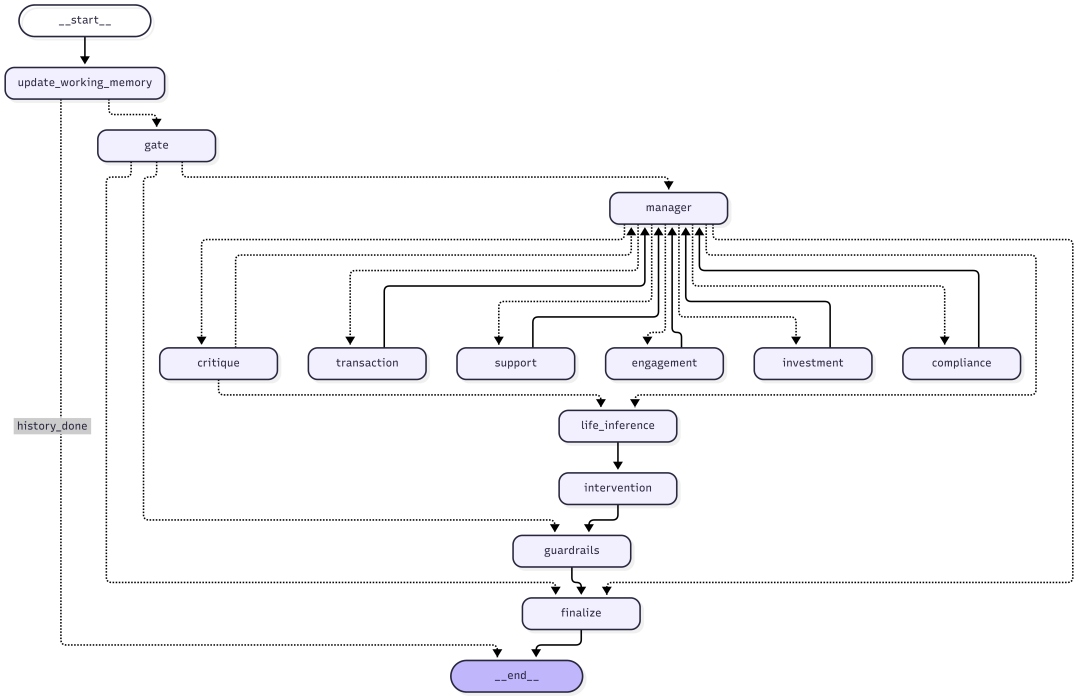


Figure 2: Compiled LangGraph (auto-exported). Solid edges are unconditional, dotted edges are conditional routes.

Life-inference and memory. The life-inference agent sees only critique-approved findings. Hypotheses must cite finding IDs, use a closed label set (16 labels), and their confidence is capped by the confidence and agent-diversity of the cited findings. The per-customer *life-phase history* (last 10 phases) and the previous decision persist in the checkpointed thread state, are read through a `life_phase_history` tool and are shown to the intervention agent, so a phase inferred earlier is still available when later events arrive. **Working memory** (`WorkingMemory`: last 80 events, last 30 findings, current decision) is scoped by customer thread, which prevents cross-customer bleed-through by construction; expiry is a fixed sliding window.

Intervention and guardrails. A fixed `LIFE_TO_STATE` map and an `ALLOWED_ACTIONS` table per state constrain the LLM: it may only pick a legal action, and acts only at high confidence or high urgency, otherwise `no_action`. Seven post-hoc guardrails then run: G1 action legal for state; G2 evidence IDs real and $\geq$2; G3 confidence floor; G4 fraud hold needs $\geq$2 distinct event kinds; G5 no offers while recent distress; G6 no re-issue of an identical open decision; G7 HITL routing (fraud hold, RM escalation, support intervention and offers always go to a human, as does anything with a hard stop, contradiction or sub-high confidence). Agents only *draft*; no code path executes an action, so blast radius is bounded by design.

HITL (`hitl/`). A pending decision creates a row in SQLite (`hitl_requests`) and blocks finalization until a reviewer runs `/approve`, `/reject` or `/modify`. Each step is appended to `hitl_audit`. `/ask` lets the reviewer query "why": the answer is generated only from the stored decision, its guardrail results and its trace, and cites reference keys (`EV:`, `RUN:`, `GR:`, ...), with a deterministic fallback if the LLM fails.

Observability. A trace bus (`observe/`) records every gate decision, manager decision, prompt, LLM response, tool call and result, finding, handoff, critique verdict, guardrail result and HITL event, keyed by (`run_id, customer_id, event_id`), to an in-memory buffer, SQLite (`trace_events`) and a JSONL file per run. The `/trace` endpoint feeds the live monitor CLI; `-replay` replays a saved run offline.

Interfaces. Three CLIs plus the server console (Fig. 3): the streamer CLI, the agent monitor, the HITL console, and the uvicorn server log. LLM: `gpt-oss-120b` on Groq with schema-constrained JSON output and three retries.

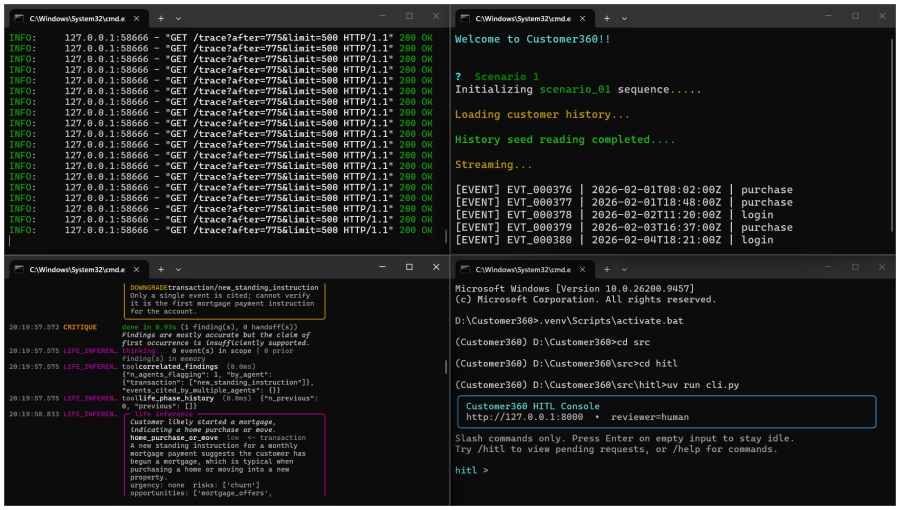


Figure 3: Running system. Top-left: server console. Top-right: streamer CLI (history seed, then live replay). Bottom-left: agent monitor (critique downgrades a single-event finding; life-inference reports `home_purchase_or_move`, low confidence). Bottom-right: HITL console.

4 Coverage Against the Production Bar

Requirement	Status	What exists / what is missing
Streaming ingestion, live transforms	**Partial**	Events are ingested one at a time over HTTP and replayed on event-time pacing; rolling windows (login trend, spend baseline, anomaly scores) are recomputed by tools over the working-memory window on each wake-up, not maintained incrementally.
Late / out-of-order events	**Partial**	Tools use a monotonic replay clock (a late event cannot rewind the window) and de-duplicate by `event_id`. There is no reordering buffer or watermark, and this was not tested against a shuffled stream.
Live vector index / RAG	**Not done**	Not implemented. No embeddings or retrieval; agents reason over structured working memory and deterministic tools.
Working memory / shared state	**Done**	Per-customer, checkpointed; agents exchange only structured findings.
Episodic and semantic memory	**Not done**	Pydantic schemas exist in `memory/schema.py` but are not wired in. Continuity across events comes only from working memory and the life-phase history. Checkpoints are in-process (`InMemorySaver`) and lost on restart.
Swarm, handoff, critique-refiner	**Done**	As described above.
Explicit agent debate	**Partial**	Disagreement is surfaced by critique and forces HITL, but there is no debate between agents.
Round robin	**Omitted by design**	No customer-facing message drafting to justify it.
Event / agent-dependent triggers	**Done**	Event-based gate; critique→life-inference→intervention and specialist handoffs are agent-dependent.
Time-based triggers	**Not done**	None. The gate's periodic review is count-based (every 15 skipped events), not scheduled.
Hard-coded stop conditions	**Done**	Gate hard stops plus seven guardrails.
Data-layer access control	**Done**	Each agent receives only events from its allowed source systems (`build_context`); other sources are not visible to it.
PII protection	**Partial**	Account IDs are masked (last 3 chars) in agent views; payload text, amounts and merchant names are not redacted in prompts or logs.
HITL: real checkpoint, audit, "why"	**Done**	Approve/reject/modify with audit table and grounded, cited Q&A.
Trace logging, prompt and tool history	**Done**	Full per-event trace, retrievable by event ID or HITL request. Metrics: per-stage latency is traced; approval/override rates and FP/FN rates are not aggregated.
Inferred-events output for scoring	**Partial**	Each decision carries the required fields (state, confidence, action, subtype, HITL status, notes) and is returned by `/events` and traced, but it is not yet exported as a standalone timestamped inferred-events file. No scoring harness was built.

5 Testing, Failure Modes and Limitations

What was run. Individual components and the graph were exercised offline and live on the scenarios, and the full four-window setup ran end-to-end. We could *not* complete a full-stream run of any scenario: the LLM provider's rate limit interrupted runs (22 agent-level and 12 event-level failures in our saved traces), so **we have no end-to-end accuracy or timeliness scores against ground truth** and cannot claim how the agents behave over a whole scenario. Observed behavior is therefore anecdotal: fraud keywords triggered hard stops and produced HITL requests

(4 in the traces), and the critique agent downgraded a single-event finding (Fig. 3).

Known weaknesses. (1) *Rate-limit fragility:* specialist, critique, life-inference and intervention failures degrade safely (no finding, deterministic checks only, `no_clear_life_event`, `no_action`), but the manager's routing call has no fallback, so a failure there aborts that event. Skipped events also mean some signals are never analyzed. (2) *Threshold tuning:* gate thresholds (e.g. 50k, 3× median) and the "act only at high confidence" rule were set by hand and not tuned against ground truth; the latter is conservative and likely delays action (timeliness). (3) *Keyword hard stops* use substring matching over the whole payload, with no negation or context handling, so false positives are possible. (4) *Memory is short-horizon:* a sliding window plus ten life phases, with no learning from HITL outcomes and no decay policy beyond truncation. (5) *Single-node state:* in-memory checkpoints and one SQLite file; not horizontally scalable.